

# BugBoard26

Documento di design del sistema

Progetto di Ingegneria del Software  
A.A. 2025/2026

|                   |                                                             |
|-------------------|-------------------------------------------------------------|
| <b>Componenti</b> | Valerio Porcile, Emanuel Giuseppe Lupoli, Emanuele Martucci |
| <b>Modalità</b>   | Tradizionale                                                |
| <b>Versione</b>   | 1.1                                                         |
| <b>Data</b>       | 2 ottobre 2026                                              |

# Indice

|           |                                                      |           |
|-----------|------------------------------------------------------|-----------|
| <b>1</b>  | <b>Introduzione</b>                                  | <b>2</b>  |
| <b>2</b>  | <b>Obiettivi di progettazione</b>                    | <b>2</b>  |
| <b>3</b>  | <b>Architettura generale</b>                         | <b>3</b>  |
| 3.1       | Component diagram . . . . .                          | 3         |
| 3.2       | Deployment diagram . . . . .                         | 4         |
| <b>4</b>  | <b>Scelte tecnologiche</b>                           | <b>5</b>  |
| 4.1       | Frontend Angular . . . . .                           | 5         |
| 4.2       | Backend ASP.NET Core Web API . . . . .               | 5         |
| 4.3       | Database PostgreSQL . . . . .                        | 5         |
| 4.4       | Autenticazione JWT . . . . .                         | 6         |
| <b>5</b>  | <b>Organizzazione del backend</b>                    | <b>6</b>  |
| 5.1       | Controller . . . . .                                 | 6         |
| 5.2       | Servizi applicativi . . . . .                        | 6         |
| 5.3       | DbContext . . . . .                                  | 6         |
| <b>6</b>  | <b>Class diagram di design</b>                       | <b>7</b>  |
| 6.1       | Vista dominio e persistenza . . . . .                | 7         |
| 6.2       | Vista servizi e repository . . . . .                 | 8         |
| <b>7</b>  | <b>Schema della persistenza dati</b>                 | <b>9</b>  |
| <b>8</b>  | <b>Principali scelte di design</b>                   | <b>10</b> |
| 8.1       | Separazione frontend e backend . . . . .             | 10        |
| 8.2       | Uso di servizi applicativi . . . . .                 | 10        |
| 8.3       | Accesso ai dati . . . . .                            | 10        |
| 8.4       | Gestione dei ruoli . . . . .                         | 10        |
| 8.5       | Suggerimento automatico dello assegnatario . . . . . | 11        |
| 8.6       | Gestione delle notifiche . . . . .                   | 11        |
| 8.7       | Gestione dei duplicati . . . . .                     | 13        |
| 8.8       | Archiviazione . . . . .                              | 13        |
| <b>9</b>  | <b>Diagrammi comportamentali</b>                     | <b>13</b> |
| 9.1       | Creazione di una issue . . . . .                     | 13        |
| 9.2       | Cambio stato con notifica . . . . .                  | 14        |
| 9.3       | Contrassegno di una issue duplicata . . . . .        | 15        |
| <b>10</b> | <b>API principali implementate</b>                   | <b>16</b> |
| <b>11</b> | <b>Testabilità</b>                                   | <b>17</b> |
| <b>12</b> | <b>Limiti e verifiche dopo il merge</b>              | <b>17</b> |
| <b>13</b> | <b>Conclusioni</b>                                   | <b>17</b> |

# 1 Introduzione

Il presente documento descrive la progettazione del sistema BugBoard26, realizzato come applicazione distribuita per la gestione collaborativa di issue in progetti software.

Il documento rappresenta la seconda fase del lavoro progettuale e riprende quanto definito nella specifica dei requisiti. In particolare, vengono descritte le scelte architetture, la suddivisione delle responsabilità tra frontend, backend e database, la struttura principale delle classi di design e le motivazioni delle principali scelte tecnologiche.

Le funzionalità considerate sono quelle assegnate al gruppo:

- autenticazione tramite email e password;
- creazione di utenti da parte degli amministratori;
- creazione di issue da parte di ADMIN e USER autenticati;
- visualizzazione, ricerca, filtro e ordinamento delle issue;
- cambio stato delle issue assegnate;
- notifica al segnalatore quando una issue viene risolta;
- esportazione delle issue in formato CSV;
- archiviazione delle issue da parte degli amministratori;
- suggerimento automatico dello assegnatario in base al carico di lavoro;
- gestione di utenti readonly;
- contrassegno di una issue come duplicata.

# 2 Obiettivi di progettazione

La progettazione del sistema è stata guidata dai seguenti obiettivi:

- separare chiaramente frontend, backend e persistenza;
- mantenere il backend indipendente dalla tecnologia utilizzata per la interfaccia utente;
- esporre le funzionalità tramite API REST;
- centralizzare la logica applicativa nel backend;
- distinguere controller, regole di dominio e persistenza tramite Entity Framework Core;
- limitare la complessità, mantenendo il sistema coerente con il lavoro di un gruppo di tre studenti;
- rendere il progetto facilmente testabile, estendibile e spiegabile in sede di discussione.

### 3 Architettura generale

Il sistema è stato progettato come applicazione web distribuita composta da tre macro componenti:

- **Frontend Angular**, responsabile della interfaccia utente e della interazione con lo utente;
- **Backend ASP.NET Core Web API**, responsabile della logica applicativa, della autenticazione, della gestione dei ruoli e della esposizione delle API REST;
- **Database PostgreSQL**, responsabile della persistenza dei dati.

La comunicazione tra frontend e backend avviene esclusivamente tramite richieste HTTP verso API REST. Le richieste protette includono un token JWT generato dal backend durante la fase di login.

Questa scelta permette di rispettare il vincolo di separazione tra frontend e backend: il backend non dipende da Angular e potrebbe essere utilizzato anche da una diversa interfaccia, per esempio una applicazione mobile o desktop.

#### 3.1 Component diagram

Il frontend contiene componenti e servizi Angular che invocano le API. Nel backend i controller usano `ApplicationDbContext`; `IssueDomainService` contiene regole di dominio e i servizi JWT e hashing gestiscono l'autenticazione. Non sono presenti repository applicativi.

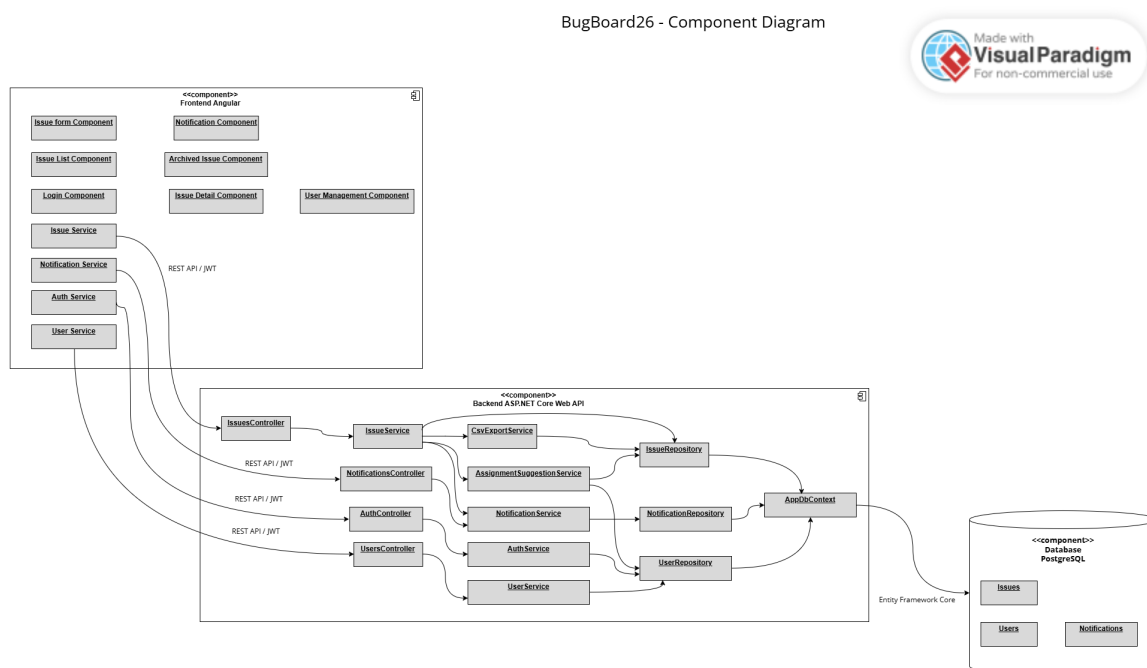


Figura 1: Component diagram della progettazione iniziale, da allineare

Il diagramma originale non è stato rigenerato. In `component_diagram.vpd` e nel PNG occorre eliminare il livello Repository, collegare i controller ad `ApplicationDbContext` e mostrare `IssueDomainService` come dipendenza di `IssuesController`. Non si aggiungono classi al codice per riprodurre il disegno precedente.

### 3.2 Deployment diagram

Il deployment diagram mostra una possibile distribuzione del sistema. Lo utente accede alla applicazione tramite browser. La parte frontend comunica con il backend tramite rete, mentre il backend accede al database PostgreSQL tramite protocollo PostgreSQL.

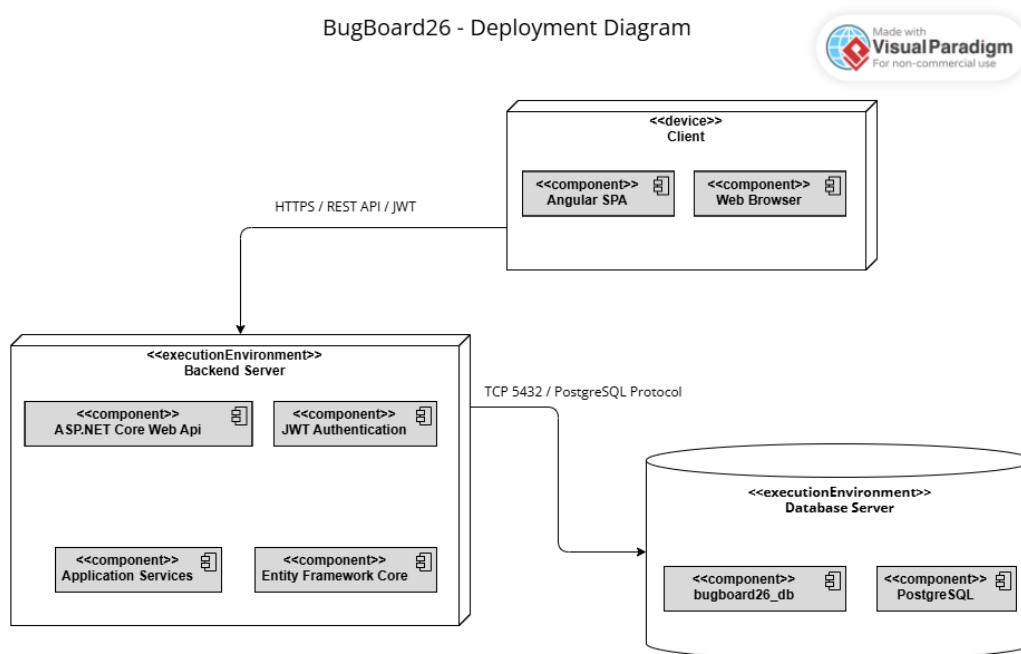


Figura 2: Deployment diagram di BugBoard26

Nell'avvio locale attuale Compose esegue PostgreSQL 16 e il backend .NET 10. Il backend usa `postgres:5432` e pubblica la porta interna 8080 su `localhost:5256`; Angular resta sul PC, sulla porta 4200. Il volume PostgreSQL esistente conserva i dati. In alternativa l'API può essere avviata con `dotnet run`, fermando prima il backend Docker. Il deployment diagram originale resta una vista logica: i dettagli locali sono in `docker/docker-compose.yml` e nel README principale.

## 4 Scelte tecnologiche

### 4.1 Frontend Angular

Angular è stato scelto per realizzare una Single Page Application strutturata in componenti. Questa tecnologia permette di separare la presentazione dalla logica di accesso alle API e offre strumenti già integrati per routing, form, servizi e gestione delle chiamate HTTP.

Nel frontend sono previsti componenti dedicati alle principali schermate:

- login;
- lista issue;
- creazione issue;
- dettaglio issue;
- archivio issue;
- notifiche;
- gestione utenti per amministratori.

### 4.2 Backend ASP.NET Core Web API

ASP.NET Core Web API è stato scelto per il backend perché supporta in modo nativo la creazione di API REST, la gestione della dependency injection, la autenticazione JWT e la integrazione con Entity Framework Core.

La struttura implementata distingue:

- **Controller**: validano richieste, applicano autorizzazioni, interrogano il contesto e restituiscono DTO;
- **IssueDomainService**: verifica i permessi di cambio stato e calcola il suggerimento dell'assegnatario;
- **ApplicationDbContext**: mappa entità e relazioni e salva le modifiche tramite Entity Framework Core;
- **Servizi di autenticazione**: `Pbkdf2PasswordHasher` e `JwtTokenService`, con le rispettive interfacce.

Questa struttura consente di separare le responsabilità e rende più semplice testare la logica applicativa.

### 4.3 Database PostgreSQL

PostgreSQL è stato scelto come database relazionale per la persistenza delle informazioni principali del sistema. Le entità principali sono utenti, issue e notifiche.

La scelta di un database relazionale è coerente con la natura dei dati, che presentano relazioni chiare tra utenti, issue e notifiche.

## 4.4 Autenticazione JWT

La autenticazione è basata su token JWT. Dopo il login, il backend restituisce un token contenente le informazioni essenziali dello utente, tra cui identificativo e ruolo. Il frontend invia il token nelle richieste successive verso le API protette.

Questa soluzione permette di mantenere il backend stateless rispetto alle sessioni e di gestire in modo semplice autorizzazioni basate sui ruoli.

## 5 Organizzazione del backend

Il backend è stato progettato separando le responsabilità in più livelli.

### 5.1 Controller

I controller rappresentano il punto di ingresso delle richieste HTTP. Ogni controller espone un gruppo di API legate a una specifica area funzionale:

- **AuthController**: login e generazione del token JWT;
- **UsersController**: creazione e consultazione degli utenti;
- **IssuesController**: creazione, ricerca, filtro, cambio stato, archiviazione, duplicati ed export;
- **NotificationsController**: consultazione e lettura delle notifiche.

### 5.2 Servizi applicativi

**IssueDomainService** lavora su oggetti del dominio, senza dipendere dal database. Espone **CanChangeIssueStatus**, **SuggestAssignee** e **SearchIssues**. La ricerca API usa invece **ILike** nelle query di **IssuesController**; l'export CSV è costruito nello stesso controller. Il controller aggiorna stato e date, aggiunge la notifica e chiama **SaveChangesAsync**. Non esistono **IssueService**, **NotificationService** o repository nel codice attuale.

### 5.3 DbContext

**ApplicationDbContext** rappresenta il punto di integrazione con Entity Framework Core e PostgreSQL. Configura indici, lunghezze dei campi, conversioni degli enum e relazioni. Espone i **DbSet**:

- **Users**;
- **Issues**;
- **Notifications**.

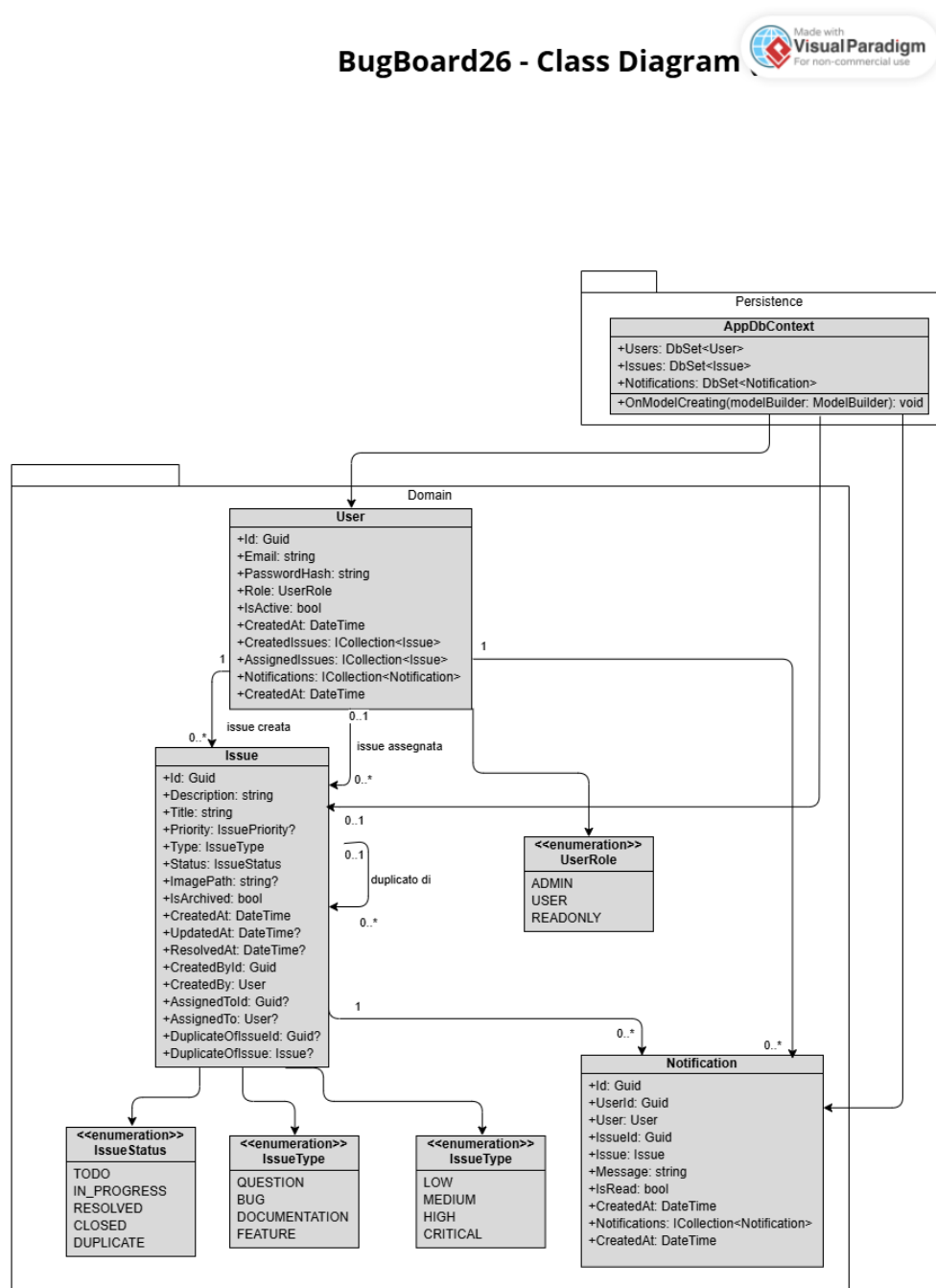
**Program.cs** chiama **DatabaseSeeder.SeedAdminAsync** prima di accettare richieste. Il seeder esegue **Database.MigrateAsync()** e crea un admin attivo solo se manca un utente di ruolo ADMIN. Le migrazioni presenti sono in **backend/BugBoard26.Api/Migrations**; non serve una seconda procedura manuale per applicarle. Connessione, JWT e credenziali iniziali sono configurabili tramite variabili d'ambiente.

## 6 Class diagram di design

Per migliorare la leggibilità, il class diagram è stato suddiviso in due viste complementari. La prima descrive il dominio e la persistenza, mentre la seconda mostra servizi, repository e dipendenze principali.

## 6.1 Vista dominio e persistenza

La vista dominio e persistenza mostra le entità principali del sistema e le relazioni tra esse.



Le entità principali sono **User**, **Issue** e **Notification**. Una issue è creata da uno utente e può essere assegnata a uno utente. Una issue può inoltre essere marcata come duplicata di un'altra issue. Le notifiche sono associate sia allo utente destinatario sia alla issue di riferimento. Nei modelli attuali tutti gli identificativi sono **int**, non **GUID**; **AssignedToId**, **DuplicateOfIssueId**, **ImagePath** e **ResolvedAt** sono opzionali. Anche questa vista storica va riallineata ai modelli, senza modificare il codice per riprodurla.

## 6.2 Vista servizi e repository

La vista servizi e repository conserva il disegno iniziale e non rappresenta le classi effettivamente implementate.

### BugBoard26 - Class Diagram

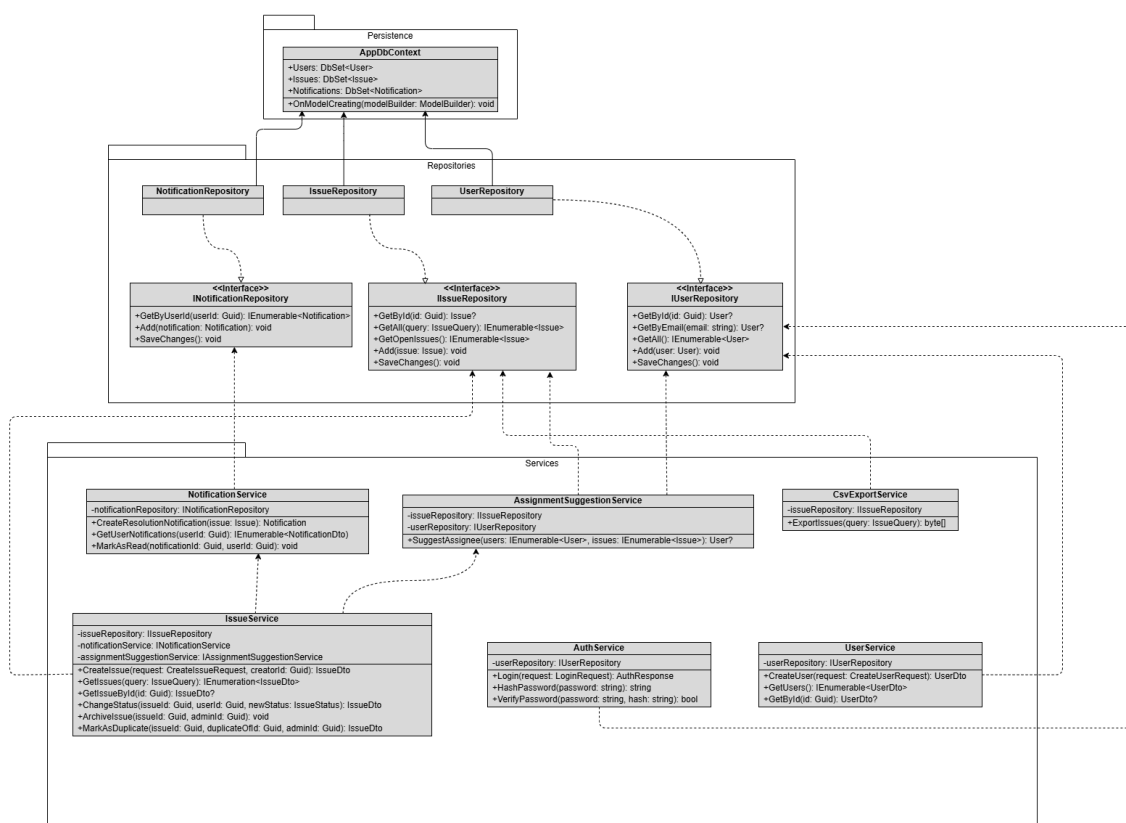


Figura 4: Class diagram iniziale: servizi e repository non implementati

In `class_diagram.vpd` e nelle immagini restano da sostituire `AppDbContext` con `ApplicationDbContext` e i servizi/repository previsti con controller, `IssueDomainService`, hasher e servizio JWT realmente presenti. Gli attributi delle entità vanno confrontati con `backend/BugBoard26.Api/Models`. Il disegno storico non implica che queste correzioni siano già state eseguite.

## 7 Schema della persistenza dati

La persistenza dei dati è basata su tre tabelle principali:

- Users;
- Issues;
- Notifications.

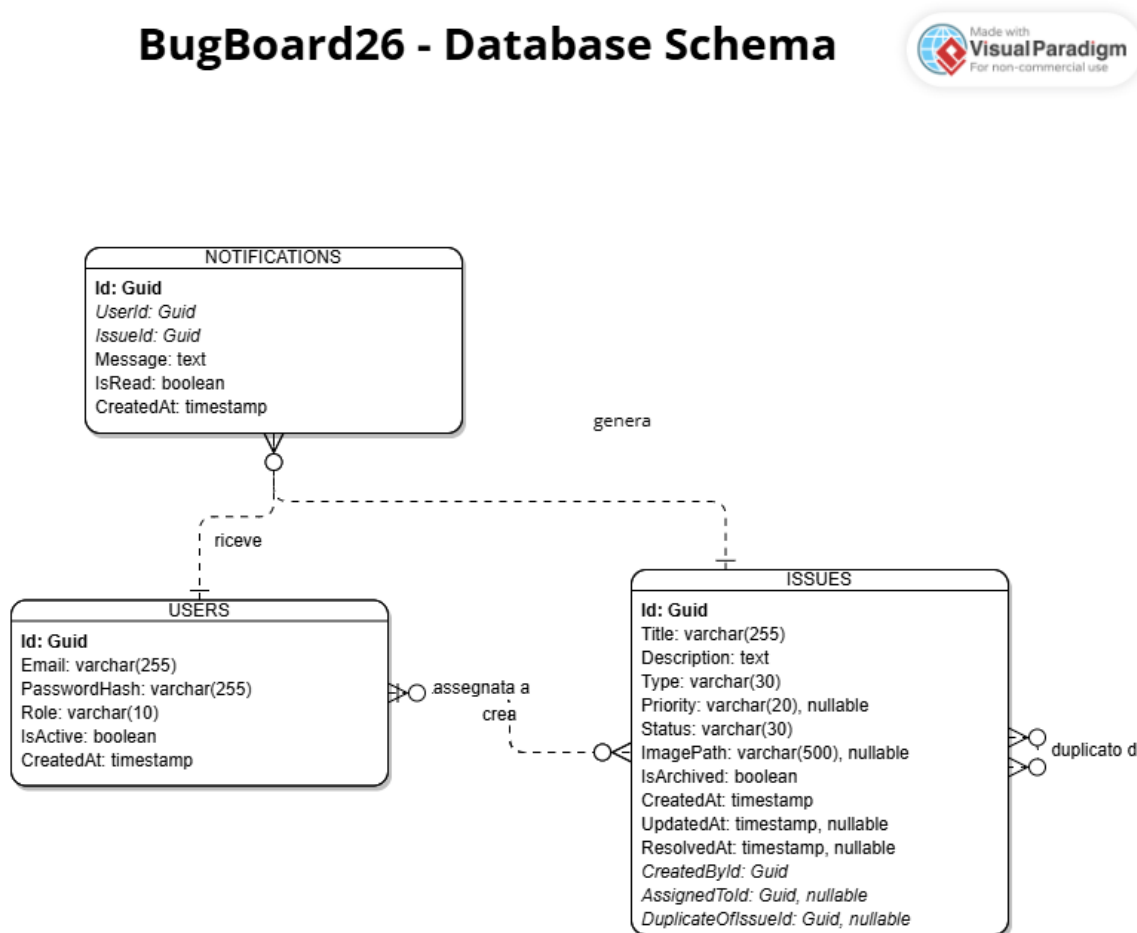


Figura 5: Schema iniziale della persistenza, da allineare alla migrazione

| Tabella       | Descrizione                                                                                                                                                                      |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Users         | Contiene gli utenti, l'hash della password, il ruolo e lo stato di attivazione. L'hashing non è cifratura reversibile.                                                           |
| Issues        | Contiene le issue create dagli utenti, il loro stato, la priorità, il tipo, lo assegnatario, le informazioni di archiviazione e lo eventuale collegamento a una issue duplicata. |
| Notifications | Contiene le notifiche generate dal sistema, in particolare quelle relative alla risoluzione di una issue.                                                                        |

`database_schema.vpd` e il PNG non sono stati rigenerati. La migrazione iniziale usa chiavi `integer`, non `GUID`, e date `timestamp with time zone`. Occorre allineare le

colonne: email 180 caratteri, hash 500, titolo 200, descrizione 4000, messaggio 500; mantenere `ImagePath` opzionale e correggere `UpdatedAt` non nullo, confrontando le relazioni con `ApplicationDbContext`. La tabella interna `__EFMigrationsHistory` registra le migrazioni applicate.

## 8 Principali scelte di design

### 8.1 Separazione frontend e backend

Il frontend non accede direttamente al database e non contiene logica di business critica. Tutte le regole principali, come controllo dei ruoli, cambio stato, archiviazione e gestione dei duplicati, sono implementate nel backend.

### 8.2 Uso di servizi applicativi

Le regole pure di `IssueDomainService` sono verificabili senza database. Le operazioni persistenti restano nei controller e sono verificate con richieste HTTP e PostgreSQL nei test di integrazione.

### 8.3 Accesso ai dati

I controller ricevono `ApplicationDbContext` tramite dependency injection. Le query di consultazione usano `AsNoTracking`; le operazioni di modifica lavorano sulle entità tracciate e chiamano `SaveChangesAsync`. Non viene introdotto un livello repository aggiuntivo.

### 8.4 Gestione dei ruoli

Il sistema prevede tre ruoli principali:

- **ADMIN:** può gestire utenti, creare issue, cambiare stato, archiviare e contrassegnare duplicati;
- **USER:** può creare issue, consultarle e cambiare lo stato delle issue assegnate;
- **READONLY:** può consultare le issue, senza crearle o modificarle.

Il controllo dei ruoli viene effettuato lato backend, in modo da impedire operazioni non autorizzate anche in caso di manipolazione della interfaccia frontend.

Lista, dettaglio, archivio, ricerca ed export CSV sono accessibili a tutti e tre i ruoli autenticati. Nel frontend `canChangeStatus` mostra il pannello solo ad ADMIN o USER assegnatario; `canCreateIssue` e `issueCreationGuard` escludono READONLY dalla creazione, mentre `adminGuard` protegge la pagina utenti. Riferimenti: `frontend/src/app/pages/issue-detail-page/issue-detail-page.component.ts`, `frontend/src/app/services/auth.guard.ts` e `frontend/src/app/app.routes.ts`.

**Requisiti del cambio stato.** RF-10 e UC-02 consentono l'operazione all'assegnatario oppure all'amministratore; RF-17 esclude readonly dalle modifiche alle issue. RNF-02 richiede il controllo sul backend e RNF-05 richiede che l'interfaccia renda chiare le operazioni disponibili per ruolo.

**Comportamento attuale.** `IssuesController.UpdateStatus` espone `PATCH /api/issues/{id}/status` per `ADMIN`, `USER`. Verifica l'account attivo e chiama `IssueDomainService.CanChangeIssueStatus`: `admin` può agire su qualsiasi issue, `USER` deve essere assegnatario e `READONLY` è escluso anche se assegnato. Una richiesta non autorizzata è rifiutata con 403; l'account corrente assente o inattivo produce 401, una issue inesistente 404.

#### Differenze e verifiche aperte.

- Il frontend propone `TODO`, `IN_PROGRESS`, `RESOLVED`, `CLOSED` e disabilita il cambio per una issue `DUPLICATE`. `UpdateStatus` non implementa una matrice di transizioni né quel blocco: non va presentata come requisito una sequenza obbligatoria.
- `UpdateStatus` accetta ancora `Duplicate` senza riferimento all'originale, anche per un `USER` assegnatario; UC-03 riserva il contrassegno all'admin e richiede l'originale. `MarkAsDuplicate` conserva l'eventuale `ResolvedAt`. Questi punti backend restano da verificare dopo il merge delle correzioni del collega: questa revisione documentale non li dichiara risolti e non modifica i requisiti.

Riferimenti: `backend/BugBoard26.Api/Controllers/IssuesController.cs`, metodi `UpdateStatus` e `MarkAsDuplicate`; `backend/BugBoard26.Api/Services/IssueDomainService.cs`, metodo `CanChangeIssueStatus`.

## 8.5 Suggerimento automatico dello assegnatario

`IssueDomainService.SuggestAssignee` considera gli utenti attivi non `READONLY`, inclusi gli `ADMIN`, e conta le issue non archiviate in stato `TODO` o `IN_PROGRESS`. Sceglie il carico minore e, a parità, l'identificativo utente minore. Questo descrive il criterio implementato, senza assumere un ordinamento per email.

## 8.6 Gestione delle notifiche

**Requisito.** RF-11 richiede la notifica al segnalatore quando la issue passa a `RESOLVED`. Per la verifica di questo flusso, ogni utente deve consultare e segnare come lette soltanto le proprie notifiche.

**Comportamento attuale della risoluzione.** In `IssuesController.UpdateStatus`, se lo stato richiesto differisce dal corrente, vengono aggiornati `Status` e `UpdatedAt`. Per `RESOLVED` viene impostato `ResolvedAt` in UTC e aggiunta una `Notification` con `UserId = CreatedById`, `IssueId`, messaggio di risoluzione, `IsRead = false` e `CreatedAt` in UTC. Una sola chiamata `SaveChangesAsync` persiste issue e notifica.

La ripetizione dello stato corrente risponde 200 con la issue, senza scritture, date nuove o ulteriori notifiche. Uscendo da `RESOLVED`, `ResolvedAt` viene azzerato senza eliminare o cambiare la lettura delle notifiche precedenti. Un nuovo ingresso in `RESOLVED` imposta una nuova data e crea una nuova notifica. Non è presente un invio email o un aggiornamento in tempo reale della pagina notifiche.

**Comportamento attuale delle notifiche.** `NotificationsController.GetAll` restituisce con `GET /api/notifications` solo le notifiche con `UserId` uguale all'utente corrente, dalla più recente. `MarkAsRead`, `PATCH /api/notifications/{id}/read`,

verifica il proprietario prima di impostare `IsRead = true`. Restituisce 403 per la notifica altrui, 404 per un ID inesistente e 401 per account corrente assente o inattivo. Anche ADMIN può operare solo sulle proprie notifiche. Ripetere la lettura di una propria notifica restituisce 200 e mantiene `IsRead = true`; non esiste un endpoint di dettaglio notifica per ID.

La pagina Angular carica l'elenco all'apertura, evidenzia le notifiche non lette e aggiorna l'elemento con la risposta del PATCH. Il pulsante *Segna come letta* è mostrato solo per notifiche non lette. Il dettaglio issue visualizza `ResolvedAt`; `IssuesService` converte i nomi di stato Angular nei nomi API, ad esempio `RESOLVED` in `Resolved`.

Riferimenti: `backend/BugBoard26.Api/Controllers/NotificationsController.cs`; modelli `backend/BugBoard26.Api/Models/Issue.cs` e `backend/BugBoard26.Api/Models/Notification.cs`; pagine `frontend/src/app/pages/notifications-page/notifications-page.component.ts` e relativo HTML; `frontend/src/app/services/notifications.service.ts`.

**Differenza da chiarire.** RF-17 parla di readonly *senza modificare dati*, mentre il design circoscrive il divieto alle issue. `NotificationsController.MarkAsRead` e la pagina notifiche non escludono `READONLY`: un account attivo di questo ruolo può segnare come letta una propria notifica. Se RF-17 vieta anche questa scrittura, il codice non è conforme; se il divieto riguarda solo le issue, il requisito va chiarito con il committente. RF-17 resta invariato e non si assume una nuova eccezione.

## 8.7 Gestione dei duplicati

Solo gli amministratori possono contrassegnare una issue come duplicata. Quando questo avviene, la issue viene collegata a una issue principale e il suo stato viene impostato a **DUPLICATE**. In questo modo rimane tracciabile ma non viene più considerata come issue aperta da risolvere.

## 8.8 Archiviazione

Solo gli amministratori possono archiviare una issue. Una issue archiviata non viene mostrata nella lista principale, ma resta accessibile da una sezione dedicata. Questa scelta permette di non eliminare dati utili e di conservare la tracciabilità delle segnalazioni.

# 9 Diagrammi comportamentali

Il diagramma di cambio stato è allineato ai controller e al contesto effettivi. I diagrammi di creazione e duplicazione conservano invece il disegno iniziale: nei rispettivi .vpd e PNG restano da eliminare i servizi/repository assenti e da rappresentare le query e il salvataggio di `IssuesController` tramite `ApplicationDbContext`.

## 9.1 Creazione di una issue

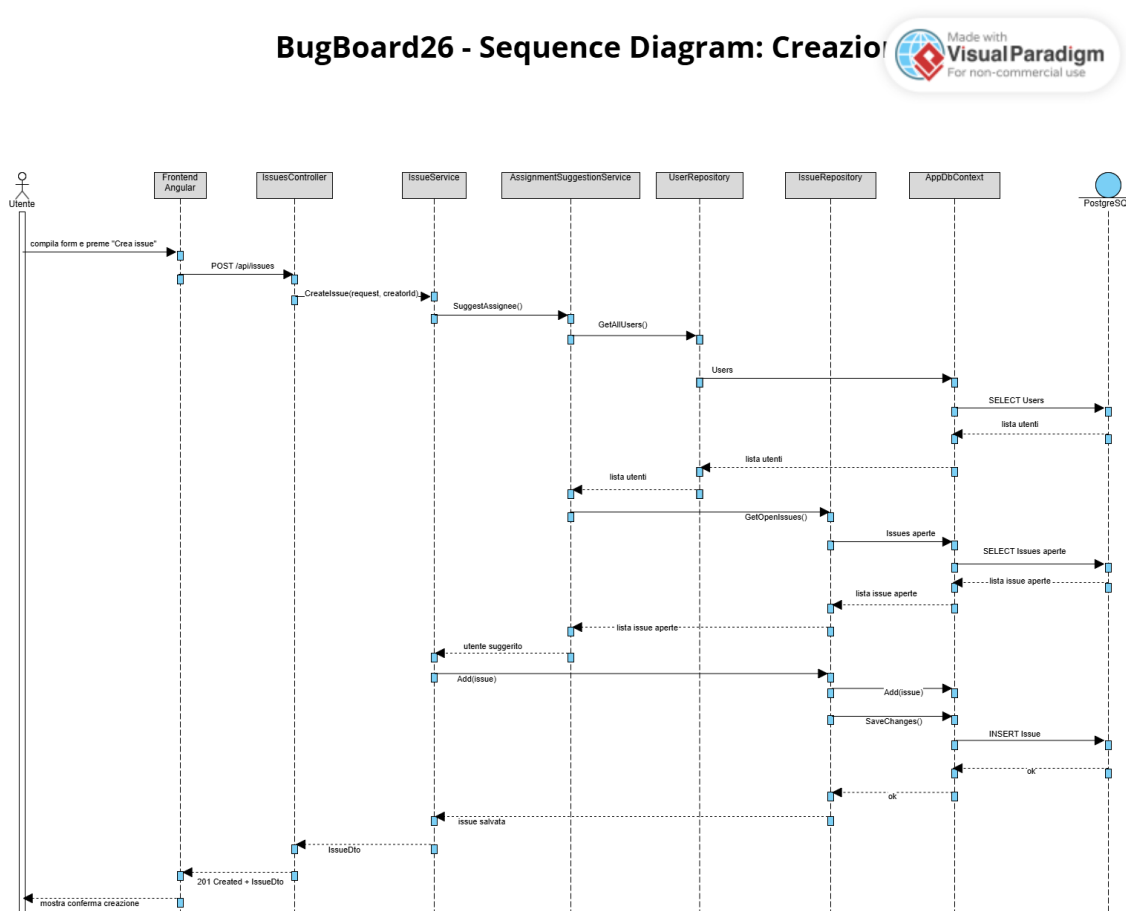


Figura 6: Sequence diagram: creazione di una issue

## 9.2 Cambio stato con notifica

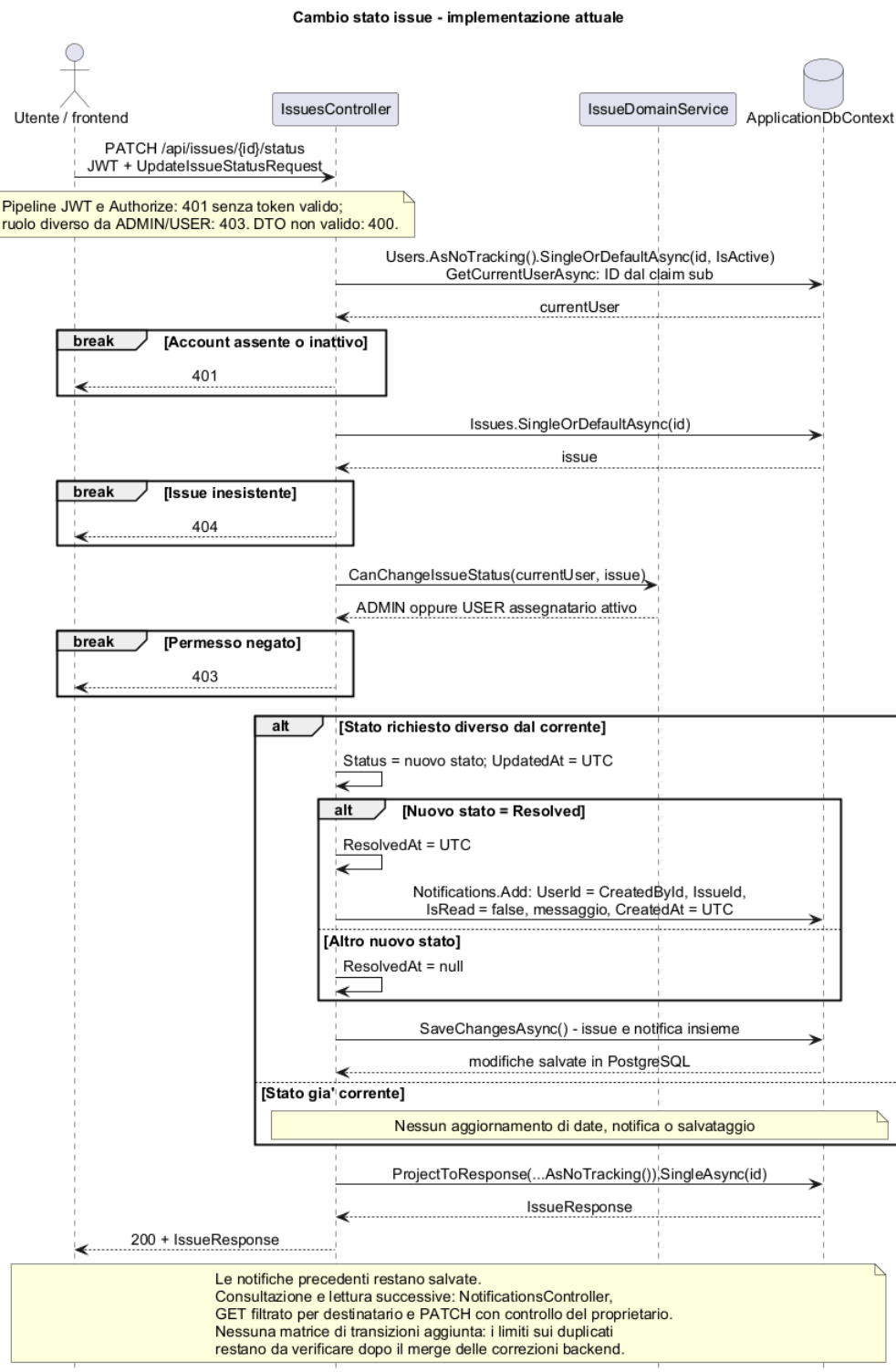


Figura 7: Cambio stato con notifica: flusso implementato

**Confronto con UC-02 e con il codice.** Il sorgente aggiornato è `sequence_cambio_stato.puml`; il PNG è generato localmente da quel sorgente con PlantUML. Il vecchio `.vpd` non è stato modificato e conserva la progettazione iniziale: non va usato per rigenerare il nuovo PNG. Il diagramma rappresenta autorizzazione, controllo dell'account attivo, caricamento della issue, `CanChangeIssueStatus`, richiesta dello stesso stato senza scritture, date, eventuale notifica e un solo `SaveChangesAsync`. La consultazione e lettura della notifica sono operazioni successive di `NotificationsController`, descritte nella sezione dedicata. Non viene introdotta una matrice di transizioni.

### 9.3 Contrassegno di una issue duplicata

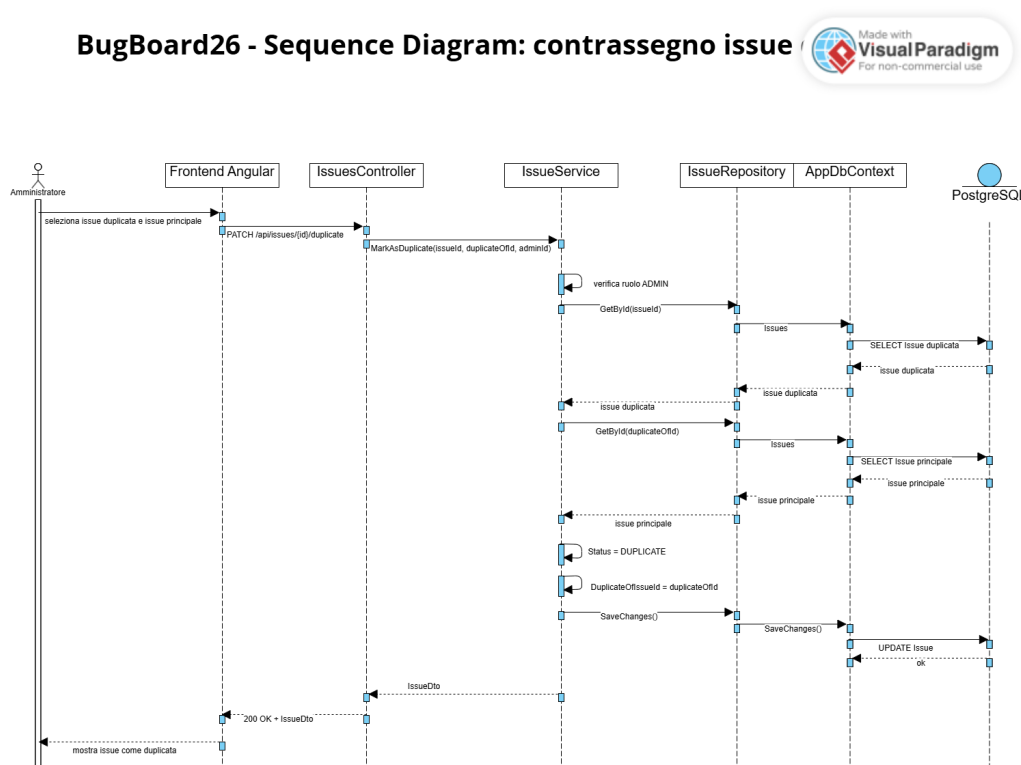


Figura 8: Sequence diagram: contrassegno di una issue duplicata

## 10 API principali implementate

| Metodo | Endpoint                                     | Descrizione                                                        |
|--------|----------------------------------------------|--------------------------------------------------------------------|
| POST   | <a href="#">/api/auth/login</a>              | Effettua il login e restituisce il token JWT.                      |
| POST   | <a href="#">/api/users</a>                   | Crea un nuovo utente. Operazione riservata agli amministratori.    |
| GET    | <a href="#">/api/users</a>                   | Restituisce la lista degli utenti; solo ADMIN.                     |
| POST   | <a href="#">/api/issues</a>                  | Crea una nuova issue; ADMIN o USER.                                |
| GET    | <a href="#">/api/issues</a>                  | Restituisce le issue con ricerca, filtri e ordinamento.            |
| GET    | <a href="#">/api/issues/{id}</a>             | Restituisce il dettaglio di una issue.                             |
| PATCH  | <a href="#">/api/issues/{id}/status</a>      | Modifica lo stato per ADMIN o USER assegnatario; READONLY escluso. |
| PATCH  | <a href="#">/api/issues/{id}/archive</a>     | Archivia una issue. Operazione riservata agli amministratori.      |
| GET    | <a href="#">/api/issues/archived</a>         | Restituisce le issue archiviate.                                   |
| PATCH  | <a href="#">/api/issues/{id}/duplicate</a>   | Contrassegna una issue come duplicata; solo ADMIN.                 |
| GET    | <a href="#">/api/issues/export</a>           | Esporta le issue in formato CSV.                                   |
| GET    | <a href="#">/api/notifications</a>           | Restituisce le notifiche dell'utente autenticato.                  |
| PATCH  | <a href="#">/api/notifications/{id}/read</a> | Segna una propria notifica come letta; 403 se appartiene ad altri. |

Tabella 2: Endpoint principali esposti dal backend

## 11 Testabilità

`backend/BugBoard26.Tests/IssueDomainServiceTests.cs` verifica regole pure di ricerca, carico e autorizzazione. I test di integrazione avviano `Program` tramite `WebApplicationFactory`, usano JWT ottenuti con il login reale, Npgsql e un PostgreSQL dedicato sulla porta 55432. Ogni caso crea un database temporaneo distinto; non usa il database di sviluppo né repository simulati.

`IssueStatusIntegrationTests` verifica risoluzione da parte dell'assegnatario con segnalatore distinto, date e notifica persistite, ADMIN non assegnatario, rifiuto di USER non assegnatario e READONLY, ripetizione di `Resolved` senza nuove date/notifiche, 401 senza token e 404 per issue assente. `NotificationsIntegrationTests` verifica elenco personale ordinato, lettura persistita per USER/ADMIN/READONLY, rifiuto di notifiche altrui, 401, 404 ed elenco vuoto. Il successo di questi test descrive il comportamento attuale, non chiarisce da solo RF-17.

La riapertura dopo `RESOLVED`, una seconda risoluzione e la ripetizione del `PATCH` read non hanno casi dedicati in questi due file: restano nella checklist manuale. Esiti realmente eseguiti e limiti sono registrati in `Documentazione/Documentazione finale/readme.md`; la compilazione LaTeX non è una prova applicativa.

## 12 Limiti e verifiche dopo il merge

- Upload immagini non implementato: `ImagePath` è presente nel modello, ma non esiste un endpoint di upload né un campo corrispondente nel DTO di creazione.
- Gestione `Duplicate` tramite `PATCH` status e `ResolvedAt` nel percorso duplicate ancora da ricontrollare dopo il merge del collega.
- RF-17 da chiarire sulla lettura delle proprie notifiche per READONLY; nessuna eccezione viene aggiunta al requisito.
- Diagrammi iniziali diversi dal cambio stato ancora da allineare come indicato nelle relative sezioni; notifiche senza email o aggiornamento realtime.

## 13 Conclusioni

La progettazione proposta divide BugBoard26 in componenti indipendenti e con responsabilità chiare. Il frontend Angular si occupa della presentazione, il backend ASP.NET Core contiene la logica applicativa e PostgreSQL gestisce la persistenza dei dati.

La struttura a livelli adottata rende il sistema più ordinato, testabile ed estendibile. Le scelte di design sono coerenti con le funzionalità assegnate e con il vincolo di realizzare un sistema distribuito con frontend e backend separati.